



HAW Hamburg

B.Sc. Information Engineering
GAAI - Generative and Agentic AI

AI-Powered Rare Disease Detection Using Next-Generation Sequencing (NGS)

Group 9

Vo Thien Hai Le	vothienhai.le@haw-hamburg.de
Chidinma Eke	chidinma.eke@haw-hamburg.de

January 23 2026

Table of Contents

1	Introduction	2
2	System Architecture	2
2.1	Ingestion: Parsing VCF Files Using <code>pysam</code>	3
2.2	Filtration: Querying the Local ClinVar Database	3
2.3	Interpretation: The RAG Agent	3
3	Implementation & Demo	4
3.1	The Agentic Workflow	4
3.2	Results & User Interface	4
4	Scalability	5

1 Introduction

In the era of industrialization and modernization, Next-Generation Sequencing has brought medical background to a new revolutionization, reducing the cost of sequencing the human genome from millions of dollars to less than \$1000. However, this technological advance has created some new challenges, which are the "interpretation bottleneck" due to the fact that there are 4 to 5 million variants relative to the reference genome in whole-genome sequencing (WGS). Therefore, clinicians usually have difficulty manually filtering variants and finding relevant literature. The current tools for filtering and interpretation often exist as separate processes.

Thus, because of this gap, a system with user-friendly interface and unification is expected, aiming at satisfying both the efficiency of raw variants input and the interpret-ability of findings using natural language. This report covers GenomAI development, the architecture, implementation, and evaluation, a Proof-of-Concept (PoC) automation process designed to solve these challenges. GenomAI makes use of the Retrieval-Augmented Generation (RAG) AI process by integrating standard bio-informatics libraries (**pysam**). The system was validated using simulated patient data representing three different rare disease profiles: Long QT Syndrome (cardiovascular) [1], Cystic Fibrosis (recessive inheritance) [2], and Gaucher Disease (metabolic) [3]. The report also discusses the scalability of the architecture, analyzing how the system can be extended to support additional disease phenotypes without having significant code modified. Patient Variant ClinVar Files (VCF) data is imported automatically and compared with the ClinVar database to determine disease markers, and a local Large Language Model (LLM) is used to provide contextual interpretations of the findings. The main objective of this project is to illustrate how an optimization of an Agentic AI in terms of diagnostic process for rare diseases makes a difference in analysis time, maintenance of data privacy through local processing.

2 System Architecture

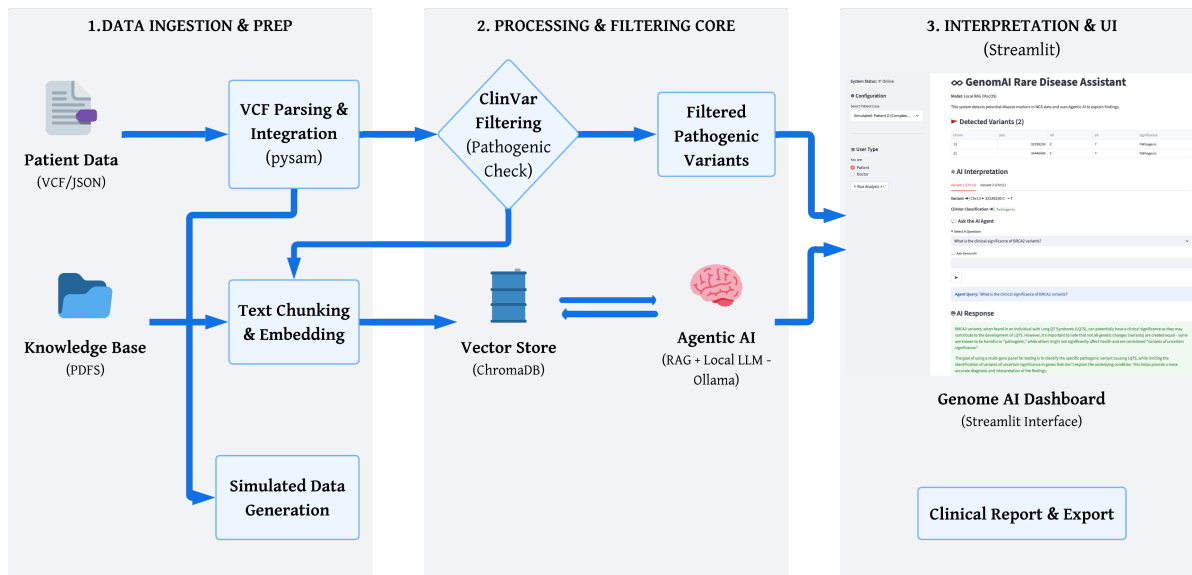


Figure 1: Project pipeline

2.1 Ingestion: Parsing VCF Files Using pysam

- **Format Standardization:** The system uses public VCF (Variant Call Format) files from NIST (National Institute of Standards and Technology) Genome in a Bottle (GIAB) [4] and NCBI (National Center for Biotechnology Information) [5], the global standard for bioinformatics, ensuring compatibility with data from any sequencer (Illumina, Oxford Nanopore, etc.)
- **Memory Efficiency:** Compressed (.vcf.gz) files are read via streaming by utilizing `pysam` library, allowing the pipeline to process files of any size line-by-line without crashing RAM.
- **Data Normalization:** Essential variant attributes, such as Chromosome (CHROM), Position (POS), Reference (REF), and Alternate (ALT) alleles are extracted and standardized to match the reference genome build (GRCh38).

2.2 Filtration: Querying the Local ClinVar Database

An optimized class `ClinVarTool` containing `get_variant_significance` function is essential for this filtering purpose and efficiency improvement.

- **Cross-Referencing:** Every patient variant is validated against a local copy of the ClinVar database. This eliminates the need for slow API calls, keeping the process offline and fast.
- **Algorithmic Speed:** The system leverages Tabix indexing (.tbi), enabling $O(1)$ (constant time) lookups. This facilitates the system to check millions of variants in seconds rather than hours.
- **Clinical Filter Logic:** A strict rule-based filter is applied to retain only variants classified as "Pathogenic" or "Likely Pathogenic", instantly reducing noise by discarding more than 99% of benign polymorphisms.

2.3 Interpretation: The RAG Agent

To read the PDF knowledge base and generate human-readable outputs, LangChain and Ollama are utilized for the RAG Agent.

1 shows the functions implemented in `tool_rag.py` to create the RAG Agent.

Table 1: Functions to create the RAG Agent.

Functions	Description
<code>setup_knowledge_base()</code>	Read PDFs from <code>knowledge_base</code> and build a Vector Database.
<code>generate_human_readable_response()</code>	Generate human-readable responses.
<code>query_literature()</code>	Search the <code>knowledge_base</code> for relevant texts.

- **Contextual Retrieval (RAG):** The system uses a **Vector Database (ChromaDB)** to store medical literature (PDFs). Once a disease is detected, relevant and specific information or paragraphs are retrieved rather than answers being hallucinated.
- **Local Inference:** A Local LLM (Mistral via Ollama) synthesizes the retrieved text to explain the variant. This ensures Zero-Data Leakage, as patient genetic data never leaves the local machine.
- **Agentic Workflow:** The AI acts as an "Expert Agent", dynamically switching context based on the specific variant selected by the user in the UI.

3 Implementation & Demo

3.1 The Agentic Workflow

Unlike standard search engines, GenomAI uses a Retrieval-Augmented Generation (RAG) approach to provide clinically relevant, evidence-based insights. The workflow operates as follows:

- **User Interaction (*Streamlit*)**: The doctor picks a patient case using the Streamlit-based UI, which initiates the automated variant analysis workflow.
- **Contextual Retrieval (*ChromaDB*)**: When a pathogenic variant is detected, the system searches a ChromaDB vector store containing indexed medical literature (PDFs) for particular evidence associated with that mutation.
- **Local Synthesis (*Ollama/Mistral*)**: A local Large Language Model (Mistral via Ollama) uses the extracted text and VCF data to create a natural language interpretation of the clinical significance.
- **Zero-Knowledge Privacy**: The agent uses local inference to ensure that sensitive genetic data never leaves the local environment, hence ensuring complete patient anonymity.
- **Dynamic Adaptation**: The AI is a "Expert Agent" that changes its environment dependent on the variation chosen. This lets it understand different situations, from BRCA2 mutations to Long QT Syndrome.

3.2 Results & User Interface

► Detected Variants (2) ⇄

chrom	pos	ref	alt	significance
13	32339230	C	T	Pathogenic
21	34449409	C	T	Pathogenic

Figure 2: The "Detected Variants" table showing the filtered hits.

Table 2: Scalability implementation.

””	Description
Database Scaling	Because ClinVar uses indexed VCFs, changing the reference file immediately adds new variations without having to redo the filtering algorithm.
Knowledge Scaling	As the RAG system leverages semantic search, introducing a new disease (e.g., Huntington’s) merely entails throwing a new PDF into the <code>knowledge_base/</code> folder.
Automatic Reindexing	The ChromaDB vector database is programmed to re-index these fresh records automatically on startup, making the new clinical context immediately accessible to the agent.
Modular Interface	The Streamlit interface dynamically updates the option menu based on available data files, allowing for a smooth transition from a few test cases to a vast rare disease library.

References

- [1] A. J. Groffen, H. Bikker, and I. Christiaans, *Long QT Syndrome Overview*. University of Washington, Seattle, Jan. 2003. [Online]. Available: https://www.ncbi.nlm.nih.gov/books/NBK1129/pdf/Bookshelf_NBK1129.pdf.
- [2] K. M. Dickinson and J. M. Collaco, “Cystic fibrosis,” *Pediatrics in Review*, vol. 42, no. 2, pp. 55–67, Feb. 2021. DOI: 10.1542/pir.2019-0212. [Online]. Available: <https://doi.org/10.1542/pir.2019-0212>.
- [3] D. A. Hughes and G. M. Pastores, *Gaucher disease*. University of Washington, Seattle, Jul. 2000. [Online]. Available: https://www.ncbi.nlm.nih.gov/books/NBK1269/pdf/Bookshelf_NBK1269.pdf.
- [4] *Genome in a bottle — NIST*, Nov. 2025. [Online]. Available: <https://www.nist.gov/programs-projects/genome-bottle>.
- [5] *National Center for Biotechnology Information*. [Online]. Available: <https://www.ncbi.nlm.nih.gov/>.

Access source code and data: GitHub source